

Turma 13 - Clusterfuss 7

Diana Sofia Sampaio Martins (up202108815) -> 50 %

Luís Carlos Ferreira Freitas (up201905767) -> 50 %

Instalação e execução:

Windows:

Executar spwin.exe

Clicar em File -> Consult... -> Selecionar o ficheiro play.pl

Na consola do SicStus: play.

Linux:

Executar SicStus Prolog

Clicar em File -> Consult... -> Selecionar o ficheiro play.pl

Na consola do SicStus: play.

Clusterfuss (Descrição do jogo):

O objetivo do Clusterfuss é remover todas as peças do adversário do tabuleiro. O jogo começa com um tabuleiro de dimensão opcional, variando entre 4x4 e 8x8, preenchido com peças vermelhas e azuis em um padrão de xadrez.

Os jogadores se alternam, começando com o jogador das peças vermelhas. Em cada turno, um jogador pode capturar uma peça adversária ou aliada, contanto que não se separe de seu próprio grupo.

As peças se movem ortogonalmente, permitindo movimentos horizontais ou verticais, mas não diagonais. Para capturar uma peça, o jogador move-se para a casa ocupada pela peça adversária e remove-a do tabuleiro.

Lógica do jogo:

Representação interna do estado de jogo:

- Tabuleiro: O tabuleiro é representado por uma lista bidimensional, sendo cada sublista uma linha do tabuleiro. Cada peça pode ter de 1 a 3 valores possíveis:

- [empty, ' '] (Representa uma posição vazia);
- [red, 'X'] (Representa uma peça vermelha com X);
- [blue, 'O'] (representa uma peça azul com O).

- Jogador atual: Os jogadores são representados pelo predicado `display_player_turn`, que acompanha os turnos dos jogadores. Após cada jogada, o tabuleiro é exibido na tela usando o predicado `display_board`, mostrando a disposição das peças ainda em jogo.

Tabuleiro inicial:

4x4:

		a	b	c	d
1		X	O	X	O
2		O	X	O	X
3		X	O	X	O
4		O	X	O	X

6x6:

		a	b	c	d	e	f
1		X	O	X	O	X	O
2		O	X	O	X	O	X
3		X	O	X	O	X	O
4		O	X	O	X	O	X
5		X	O	X	O	X	O
6		O	X	O	X	O	X

8x8:

		a	b	c	d	e	f	g	h
1	X	O	X	O	X	O	X	O	
2	O	X	O	X	O	X	O	X	
3	X	O	X	O	X	O	X	O	
4	O	X	O	X	O	X	O	X	
5	X	O	X	O	X	O	X	O	
6	O	X	O	X	O	X	O	X	
7	X	O	X	O	X	O	X	O	
8	O	X	O	X	O	X	O	X	

display_human_turn - Este predicado faz a interação entre os jogadores humanos de forma a obter a posição da peça que pretende mover, e posição para a qual se pretende que a peça seja movida.

display_game - Este predicado tem como função ir atualizando o tabuleiro do jogo à medida que as moves vão acontecendo.

Visualização do estado de jogo:

Após executarmos o jogo com o predicado `play`, o jogador tem acesso a um menu inicial com 5 opções do jogo.

Para seleccionar uma das opções disponíveis, é necessário introduzir o número correspondente à ação desejada.

As primeiras 3 opções correspondem a tipos de jogo disponíveis:

- 1 - Human vs Human
- 2 - Human vs Computer
- 3 - Computer vs Computer

Assim que uma das primeiras 3 opções do jogo é escolhida, é apresentado o tabuleiro do jogo conforme a dimensão especificada, e iniciado o loop do jogo.

A 4ª opção corresponde à opção "Game description", que contém uma breve explicação do jogo.

Por último, a 5ª opção corresponde à opção "Quit", que é utilizada para encerrar o jogo.

Ao escolher uma das opções do jogo o tabuleiro inicial é escrito no ecrã (`initial_state`), e depois vai sendo atualizado ao longo do jogo, a cada jogada, usando o predicado `display_game`.

`initial_state` - Este predicado tem como função desenhar o tabuleiro inicial do jogo.

`display_game` - Este predicado tem como função ir atualizando o tabuleiro do jogo à medida que as moves vão acontecendo.

```
***** Welcome to the game of Clusterfuss! *****
*
*               Input               Option
*
*               1               Human vs Human
*               2               Human vs Computer
*               3               Computer vs Computer
*               4               Game description
*               0               Quit
*
*****
Choose an option:
```

```
initial_state(Size, (Board, r)) :-  
    display_top_row(Size),  
    initial_board(Size, Board),  
    display_board_with_left_column(Board, 1, Size).
```

```
display_game((NewBoard, NewPlayer)) :-  
    length(NewBoard, Size),  
    display_top_row(Size),  
    display_board_with_left_column(NewBoard, 1, Size).
```

Execução de jogadas:

Em cada jogada, é apresentado o turno do jogador, com a opção de selecionar qual a peça do tabuleiro deseja mover, devendo este responder com a letra da coluna e o número da linha pretendidos.

Se o jogador inserir uma posição inválida, seja da peça a ser movida ou da posição final do movimento, uma mensagem de erro será exibida, e o jogador deverá escolher uma posição válida. Se o movimento for válido, a jogada será realizada com sucesso, e a peça escolhida será movida para a posição desejada.

display_human_turn - Este predicado faz a interação entre os jogadores humanos de forma a obter a posição da peça que pretende mover, e posição para a qual se pretende que a peça seja movida.

move - Este predicado é a regra principal responsável pela realização dos movimentos das peças no jogo.

```
move((Board, Player), Move, (NewBoard, NewPlayer)) :-  
    display_human_turn((Board, Player), Move, Board, NewBoard).
```

```
display_human_turn((Board, Player), (Row1, Col1, Row2, Col2), Board, NewBoard) :-  
    length(Board, Size),  
    nl,  
    (Player = r -> Symbol = 'X'; Player = b -> Symbol = 'O'),  
    format('Player ~w. It\'s your turn!~n', [Symbol]), nl,  
    write('About the piece that you want to move: '), nl,  
  
    write('- Column (ex: a) '), read(Col1Char),  
    validate_column(Board, Col1Char),  
  
    write('- Row (ex: 1) '), read(Row1),  
    validate_row(Board, Row1),  
  
    format_input(Row1, Col1Char, Row1Dec, Col1),  
    validate_player(Board, Row1Dec, Col1, Player, NewRow1Dec, NewCol1),  
  
    move_piece(Board, NewRow1Dec, NewCol1, e, AuxBoard),  
  
    write('About the position that you want to move to: '), nl,  
  
    write('- Column (ex: b) '), read(Col2Char),  
    validate_column(AuxBoard, Col2Char),  
  
    write('- Row (ex: 2) '), read(Row2),  
    validate_row(AuxBoard, Row2),  
  
    format_input(Row2, Col2Char, Row2Dec, Col2),  
  
    validate_not_empty_position(AuxBoard, Row2Dec, Col2, NewRow2Dec, NewCol2),  
  
    validate_orthogonal_move(Board, AuxBoard, NewRow1Dec, NewCol1, NewRow2Dec, NewCol2, FinalRow2Dec, FinalCol2),  
  
    move_piece(AuxBoard, FinalRow2Dec, FinalCol2, Player, NewBoard),  
  
    check_win(NewBoard, Player).
```

Lista de jogadas válidas:

Relativamente à seleção da peça que pretendemos mover, essa escolha é válida quando se trata de uma peça da cor com que estamos a jogar (`validate_player`).

No que diz respeito à posição para onde vamos mexer a peça escolhida, a validação é mais extensa. 1) O jogador é obrigado a mover-se para uma posição em que esteja outra peça (`validate_not_empty_position`). 2) O movimento tem que ser ortogonal e de apenas uma casa (`validate_orthogonal_move`). 3) Após a jogada, a peça em questão não se pode separar do grupo (`validate_separating`).

`validate_player` - Este predicado tem como função garantir de que a peça selecionada no seu turno não corresponde a uma peça da cor adversária.

`validate_not_empty_position` - Este predicado tem como função garantir que a posição escolhida não é uma célula vazia.

`validate_row` e `validate_col` - Estes predicados têm como função verificar se as linhas e colunas inseridas são válidas - se estão dentro dos limites do tabuleiro.

`validate_separating` - Este predicado tem como função garantir que a peça escolhida não se separa do grupo aquando do movimento.

`is_valid_move` - Este predicado verifica se os movimentos são válidos.

`valid_moves` - Este predicado define a lista das posições válidas nas quais as peças se podem mover.

`all_valid_moves` - Este predicado procura todos os movimentos válidos para as peças no tabuleiro do jogo.

```
validate_column(Board, Col1Char) :-
    char_code(Col1Char, Col1Code),
    Col1Code >= 97,
    length(Board, Size),
    Col1Code < 97 + Size.

validate_column(Board, Col1Char) :-
    nl,
    write('Invalid column! Choose a valid letter. '), nl,
    write('- Column (ex: a) '), read(NewCol1Char),
    validate_column(Board, NewCol1Char).

validate_row(Board, Row1) :-
    Row1 > 0,
    length(Board, Size),
    Row1 <= Size.

validate_row(Board, Row1) :-
    write('Invalid row! Choose a valid number. '), nl,
    write('- Row (ex: 1) '), read(NewRow1),
    validate_row(Board, NewRow1).
```



```

validate_separating(Board, Row, Col) :-
    AuxColPlus is Col + 1,
    AuxColMinus is Col - 1,
    AuxRowPlus is Row + 1,
    AuxRowMinus is Row - 1,
    (
        is_empty(Board, Row, AuxColPlus), is_empty(Board, Row, AuxColMinus), is_empty(Board, AuxRowPlus, Col), is_empty(Board, AuxRowMinus, Col) ->
        nl,
        write('Invalid move! Choose a move that doesn\'t separate from the rest.'), nl,
        write('- Column (ex: b) '), read(NewCol2Char),
        validate_column(Board, NewCol2Char),
        write('- Row (ex: 2) '), read(NewRow2),
        validate_row(Board, NewRow2),
        format_input(NewRow2, NewCol2Char, NewRow2Dec, NewCol2),
        validate_separating(Board, NewRow2Dec, NewCol2);
        true
    ).

```

```

validate_player(Board, Row1Dec, Col1, Player, NewRow1Dec, NewCol1) :-
    write('Invalid player! Choose a piece of your color.'), nl,
    write('- Column (ex: a) '), read(NewCol1Char),
    validate_column(Board, NewCol1Char),
    write('- Row (ex: 1) '), read(NewRow1),
    validate_row(Board, NewRow1),
    format_input(NewRow1, NewCol1Char, NewRow1Dec, NewCol1),
    validate_player(Board, NewRow1Dec, NewCol1, Player, FinalRow1Dec, FinalCol1).

validate_not_empty_position(Board, Row2Dec, Col2, Row2Dec, Col2) :-
    nth0(Row2Dec, Board, Row),
    (nth0(Col2, Row, r); nth0(Col2, Row, b)).

validate_not_empty_position(Board, Row2Dec, Col2, NewRow2Dec, NewCol2) :-
    nl,
    write('Invalid move! Choose a position that is not empty.'), nl,
    write('- Column (ex: b) '), read(NewCol2Char),
    validate_column(Board, NewCol2Char),
    write('- Row (ex: 2) '), read(NewRow2),
    validate_row(Board, NewRow2),
    format_input(NewRow2, NewCol2Char, NewRow2Dec, NewCol2),
    validate_not_empty_position(Board, NewRow2Dec, NewCol2, NewRow2Dec, NewCol2).

validate_orthogonal_move(Board, AuxBoard, Row1Dec, Col1, Row2Dec, Col2, Row2Dec, Col2) :-
    (Row1Dec == Row2Dec ->
        (Col1 == Col2 + 1; Col1 == Col2 - 1));
    (Col1 == Col2 ->
        (Row1Dec == Row2Dec + 1; Row1Dec == Row2Dec - 1)).

validate_orthogonal_move(Board, AuxBoard, Row1Dec, Col1, Row2Dec, Col2, NewRow2Dec, NewCol2) :-
    nl,
    write('Invalid move! Choose an orthogonal move.'), nl,
    write('- Column (ex: b) '), read(NewCol2Char),
    validate_column(AuxBoard, NewCol2Char),
    write('- Row (ex: 2) '), read(NewRow2),
    validate_row(AuxBoard, NewRow2),
    format_input(NewRow2, NewCol2Char, NewRow2Dec, NewCol2),
    validate_orthogonal_move(Board, AuxBoard, Row1Dec, Col1, NewRow2Dec, NewCol2, NewRow2Dec, NewCol2).

```


Fim do jogo:

O fim do jogo é verificado com recurso aos predicados `check_win`, `player_has_won` e `game_over`, que determinam se algum dos jogadores venceu ao fim de um loop. Para isto, o jogo é analisado para confirmar se todas as peças adversárias foram removidas ou se não há mais movimentos possíveis. `game_over` identifica o jogador vencedor com base no que os predicados anteriores retornam.

player_has_won - Este predicado tem como função verificar se algum dos jogadores é vencedor, tanto por todas as peças adversárias terem sido eliminadas do tabuleiro, quer por um jogador já não ter mais moves válidas.

game_over - Este predicado tem como função identificar o jogador vencedor e terminar o loop do jogo.

```
check_win((Board, Player)) :-  
    custom_flatten(Board, FlatBoard),  
    (player_has_won(FlatBoard, Player) ->  
        game_over((Board, Player));  
        nl  
    ).  
  
% Predicate to check if a player has won  
player_has_won(Board, Player) :-  
    all_empty_or_player(Board, Player).
```

```
game_over((Board, Player)) :-  
    (Player = r -> Symbol = 'X'; Player = b -> Symbol = 'O'),  
    nl,  
    format('Player ~w has won! Congratulations!~n', [Symbol]),  
    sleep(3),  
    halt.
```

Avaliação do tabuleiro:

Para avaliar o tabuleiro, criamos alguns predicados cuja utilidade aproveitamos ao máximo.

`all_empty_or_player` analisa se todas as células (Linha, Coluna) do tabuleiro estão, ou vazias, ou preenchidas apenas por peças pertencentes ao jogador. Este predicado tem o intuito de verificar se estamos perante uma vitória ou não.

O `replace_piece` permite substituir uma peça numa posição específica do tabuleiro, algo que é necessário no fim da execução de cada jogada.

`all_empty_or_player` - Este predicado permite perceber se todos os elementos numa lista são vazios ou pertencem ao jogador.

`replace_piece` - Este predicado permite substituir uma peça por outra, dada numa posição específica no tabuleiro.

```
% Predicate to check if all elements in a list are either 'e' or belong to the given Player
all_empty_or_player([], _).
all_empty_or_player(['e' | Rest], Player) :-
    all_empty_or_player(Rest, Player).
all_empty_or_player([Player | Rest], Player) :-
    all_empty_or_player(Rest, Player).
```

```
replace_piece([_|T], 0, X, [X|T]).
replace_piece([H|T], I, X, [H|R]) :-
    I > 0,
    I1 is I - 1,
    replace_piece(T, I1, X, R).
```

Jogada do computador:

Implementar a jogada do computador foi um dos maiores desafios, pois o Clusterfuss é um jogo com muitas restrições nos movimentos de peças a cada jogada. Portanto, aoenas conseguimos implementar apenas a jogada aleatória do computador. Começamos por identificar todas as peças que o jogador pode mover (suas peças). Em seguida, escolhemos aleatoriamente uma dessas peças e avaliamos os seus movimentos válidos possíveis. Se houver uma jogada válida, o computador realiza-a.

move2 - Este predicado tem como função permitir que o jogo flua entre um jogador e o computador.

move3 - Este predicado tem como função colocar dois computadores a jogar aleatoriamente um com o outro.

```
move2((Board, Player), Move, (NewBoard, NewPlayer)) :-  
    display_player_turn2((Board, Player), Move, Board, NewBoard).  
  
move3((Board, Player), Move, (NewBoard, NewPlayer)) :-  
    display_computer_turn((Board, Player), Move, Board, NewBoard).
```

```
display_computer_turn((Board, Player), (Row1, Col1, Row2, Col2), Board, NewBoard) :-  
    possible_pieces(Board, Player, PossiblePieces),  
    other_player(Player, OtherPlayer),  
    (PossiblePieces = [] -> game_over((Board, OtherPlayer)), !; true),  
    all_valid_moves(Board, Player, PossiblePieces, AllValidMoves),  
  
    (AllValidMoves = [] -> game_over((Board, OtherPlayer)), !;  
  
        random_piece((Row, Col), PossiblePieces),  
        valid_moves(Board, Player, (Row, Col), PossibleMoves),  
  
        [PossibleMoves = [] -> display_computer_turn((Board, Player), (Row1, Col1, Row2, Col2), Board, NewBoard);  
        FinalRow is Row + 1,  
        FinalCol is Col + 97,  
        char_code(Char, FinalCol), nl,  
  
        (Player = r -> Symbol = 'X'; Player = b -> Symbol = 'O'), nl,  
        format('Player ~w is thinking...', [Symbol]), nl, nl,  
        wait(0.5),  
  
        random_move((Row2, Col2), PossibleMoves),  
        FinalRow2 is Row2 + 1,  
        FinalCol2 is Col2 + 97,  
        char_code(Char2, FinalCol2),  
        write('Moving '), write(FinalRow), write(Char), write(' to '), write(FinalRow2), write(Char2), write('. '), nl,  
  
        move_piece(Board, Row, Col, e, TempBoard),  
        move_piece(TempBoard, Row2, Col2, Player, NewBoard),  
        check_win((NewBoard, Player)),  
        wait(0.5)
```

Conclusões:

Este projeto teve como principal objetivo cimentar os conhecimentos adquiridos sobre a linguagem de programação Prolog, no âmbito da unidade curricular de PFL.

As principais dificuldades que enfrentamos foram relacionadas com a adaptação à linguagem, especialmente no que diz respeito à realização de condicionais em Prolog.

Possíveis melhorias:

Destacamos como possível melhoria a jogabilidade do computador, uma vez que a jogada que não conseguimos desenvolver o modo greedy. Para além disso, a lógica de jogo não está totalmente igual à do Clusterfuss original, algo que também consideramos complicado de desenvolver.

Concluindo, consideramos que finalizamos o projeto com sucesso e melhoramos consideravelmente o nosso conhecimento acerca de Prolog durante a realização do mesmo.

Bibliografia:

http://marksteeregames.com/Clusterfuss_rules.pdf

<https://sicstus.sics.se/sicstus/docs/latest4/html/sicstus.html/>